

Database Management System

A. Hemalatha Reddy
Assistant Professor
CSE-Data Science

- ▶ **Data:** Raw, unprocessed facts, numbers, or observations lacking inherent meaning (e.g., "101", "Smith").
- ▶ **Information:** Processed data organized into a meaningful, usable context (e.g., "Student ID 101 belongs to Smith").
- ▶ **Database:** A database is an organized collection of data that can be easily accessed, managed, and updated.
- ▶ A **Database Management System (DBMS)** is system software used to **efficiently define, create, maintain, and securely share** structured databases among multiple users and applications.

Need for DBMS

- ▶ Efficient data storage and retrieval.
- ▶ Reduces data redundancy.
- ▶ Ensures data consistency.
- ▶ Provides data security.
- ▶ Supports multi-user access.

Purpose of database

- ▶ Data storage and management
- ▶ Data retrieval
- ▶ Backup and recovery
- ▶ Security management
- ▶ Transaction processing
- ▶ Concurrency control

Advantages of DBMS

- ▶ Reduced data redundancy
- ▶ Improved data sharing
- ▶ Better data security
- ▶ Data integrity and consistency
- ▶ Faster decision-making

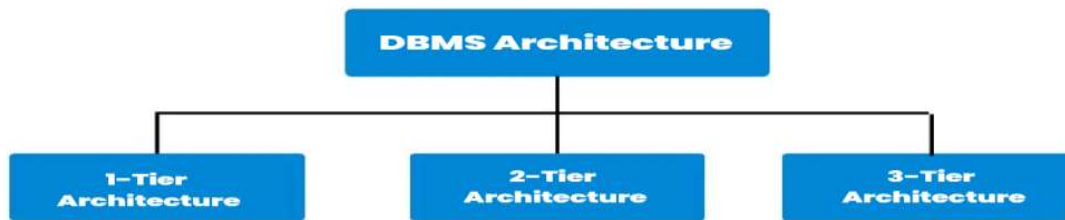
Examples of DBMS

- ▶ MySQL
- ▶ Oracle Database
- ▶ Microsoft SQL Server
- ▶ MongoDB

Applications of DBMS

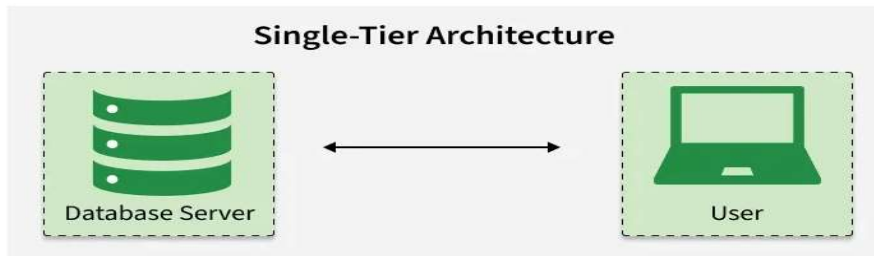
- ▶ Banking systems
- ▶ Healthcare management
- ▶ E-commerce websites
- ▶ Educational institutions
- ▶ Airline reservation systems

Database architecture



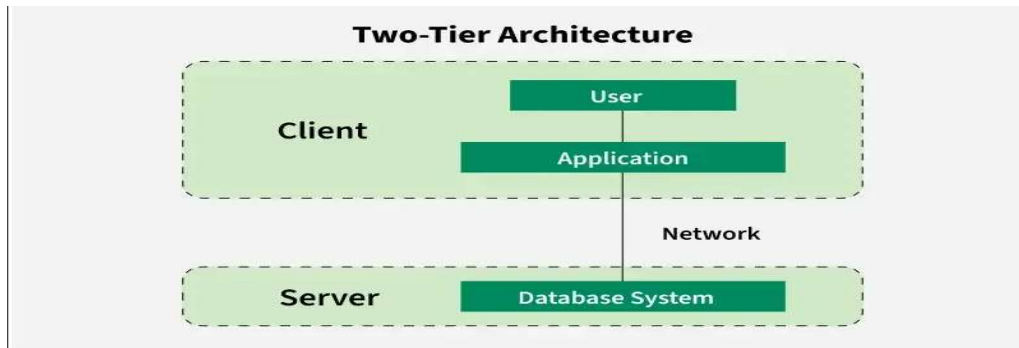
► 1-Tier Architecture

- 1-Tier Architecture: the user works directly with the database on the same system. This means the client, server, and database are all in one application. The user can open the application, interact with the data, and perform tasks without needing a separate server or network connection.



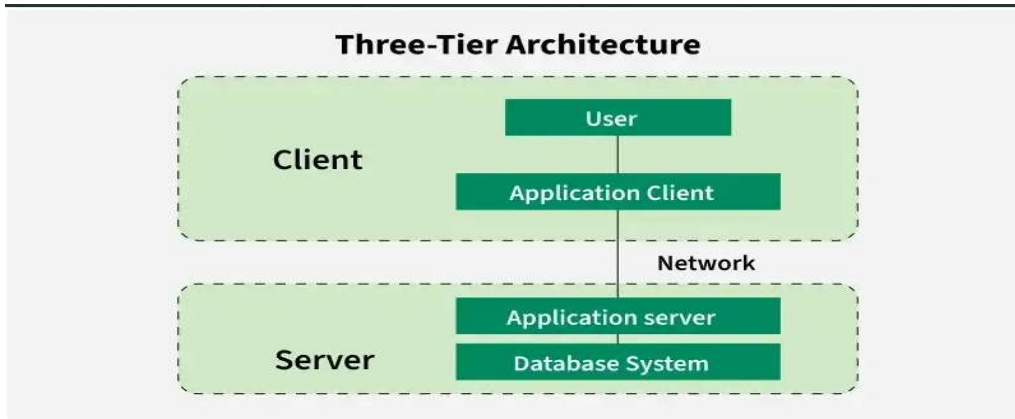
2-Tier Architecture

- The 2-tier architecture is similar to a basic [client-server model](#). The application at the client end directly communicates with the database on the server side. APIs like ODBC and JDBC are used for this interaction. The server side is responsible for providing query processing and transaction management functionalities.

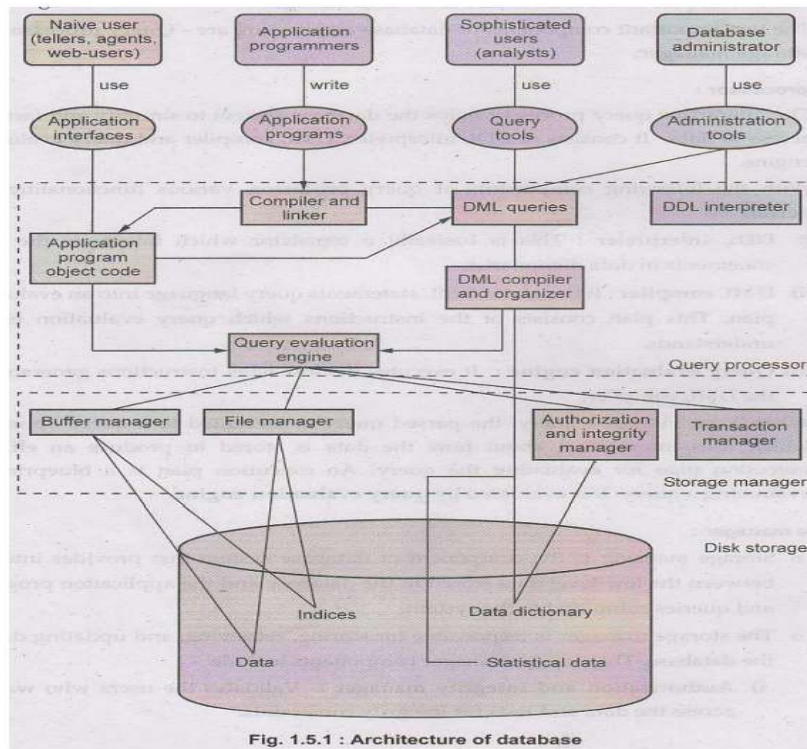


3-Tier Architecture

- In 3-Tier Architecture, there is another layer between the client and the server. The client does not directly communicate with the server. Instead, it interacts with an application server which further communicates with the database system and then the query processing and transaction management takes place. This intermediate layer acts as a medium for the exchange of partially processed data between the server and the client. This type of architecture is used in the case of large web applications.



database architecture and structure

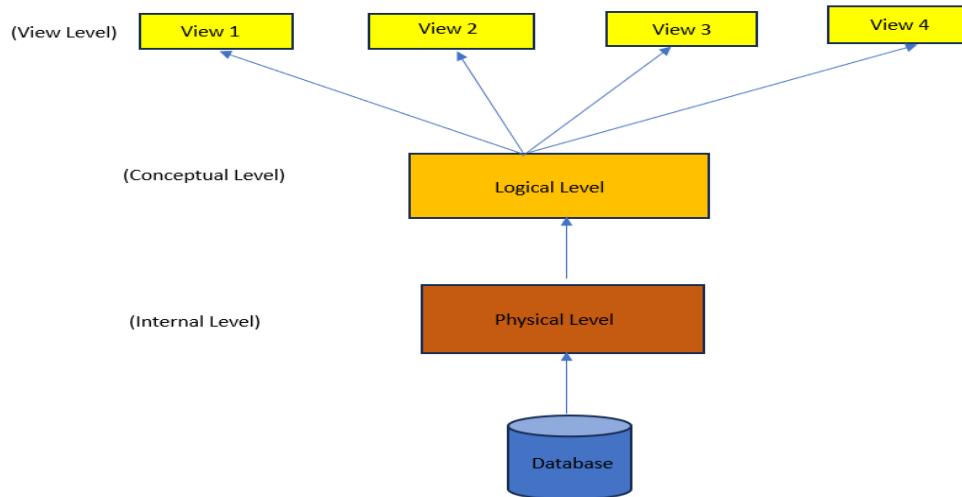


What is Data Abstraction in DBMS?

- ▶ Data Abstraction is one of the most important concepts in DBMS. Data abstraction is the process of hiding unwanted and irrelevant details from the end user. It helps to store information in such a way that the end user can access data which is necessary, the user will not be able to see what data is stored or how it is stored in a database.
- ▶ Data abstraction hides unwanted details, showing users only relevant data.
- ▶ It keeps data safe by hiding how and where data is stored.
- ▶ Abstraction simplifies access while maintaining data security and efficiency.
- ▶ For example, while buying clothes, users see only color, size, and brand, not production details.

Levels of Abstraction in DBMS

- ▶ There are three levels of data abstraction in DBMS that are mentioned below:
 - ▶ Physical or Internal level
 - ▶ Logical or Conceptual level
 - ▶ View or External level
-



Physical or Internal Level

- ▶ It is the lowest level of data abstraction which defines how data is stored in database . It defines data structures used to store data and methods to access data in database. It is very complex to understand and hence kept hidden from user. Database administrator decides how and where to store the data in database.
- ▶ Physical level deals with actual storage details like data organization, disk space allocation and data access methods.

Logical or Conceptual Level

- ▶ It is intermediate level present next to physical level. It defines what data is present in database and their relationships between them . It is less complex as compared to physical level. Programmers generally work at this level and depending on data, structure of tables, relationships and their constraints is decided at this level.

View or External Level

- ▶ It is the highest level in abstraction. There are different levels of views and each view defines only a part of whole data required to user. This level defines many views of same database for simplification of view to user. This is the highest level and easiest to understand for user.

What is Data Independence in DBMS?

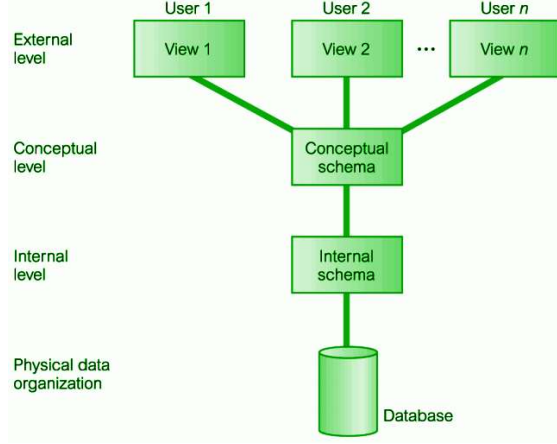
- ▶ Data Independence is a fundamental concept in Database Management Systems (DBMS) that refers to the ability to modify the schema at one level of the database without affecting the schema at the next higher level. This concept ensures that changes in how data is stored or structured internally do not impact how users or applications access and interact with the data.

Why is Data Independence Important?

- ▶ **Reduces Maintenance:** Developers don't need to update applications every time the database structure changes.
- ▶ **Increases Flexibility:** The database can be reorganized or optimized internally without affecting user queries.
- ▶ **Supports Long-Term Growth:** As business needs evolve, the database can be updated without breaking existing systems.

DBMS Three-Level Architecture

- ▶ To understand data independence, it's essential to know how DBMS is organized:
- ▶ **Internal Level :** Deals with physical storage (files, indexing, compression).
- ▶ **Conceptual Level :** Describes structure like tables, fields and relationships.
- ▶ **View Level :** Defines how users and applications see the data.



- Data independence exists between these levels:
- **Physical Data Independence:** Change in the internal level without affecting the logical level.
- **Logical Data Independence:** Change in the logical level without affecting the view level.

Logical Data Independence

- ▶ Ability to change the logical structure (tables, columns, relationships) without affecting external views or application programs. Purpose of this to allow the database structure to evolve without impacting user access or requiring changes in application code.
- ▶ **Why it matters:**
 - ▶ Helps modify the structure of the database as business needs evolve.
 - ▶ Applications and user interfaces remain unaffected.
- ▶ **Example:**
 - ▶ Adding a column like email in the employees table.
 - ▶ Creating a new relationship between two tables.
 - ▶ Merging two tables into a view for simplified access.
- ▶ **Benefits:**
 - ▶ Easier maintenance of application code.
 - ▶ Allows smoother updates in growing systems.
 - ▶ Helps support new business requirements without rewriting existing queries.

Physical Data Independence

- ▶ The ability to change how data is physically stored without affecting the logical schema or user-facing applications. Purpose is to improve performance, storage efficiency, or hardware configurations without changing how the data is structured logically.
- ▶ **Why it matters:**
 - ▶ Enables performance tuning and hardware changes
 - ▶ Does not affect how data is structured or queried
- ▶ **Example:**
 - ▶ Moving data files from the C: drive to the D: drive.
 - ▶ Creating an index to speed up queries.
 - ▶ Switching from HDD to SSD for better performance.
 - ▶ Compressing data files to save space.
- ▶ **Benefits:**
 - ▶ Backend optimizations without affecting users
 - ▶ Reduces need for structural changes during storage upgrades
 - ▶ Improves long-term maintainability

Introduction to Database languages

- ▶ Database languages are used to define, manipulate, control, and query data in a database.
- ▶ They help users interact with a Database Management System (DBMS).
- ▶ Most database languages are implemented through SQL (Structured Query Language).

Types of Database Languages

- ▶ Data Definition Language (DDL)
- ▶ Data Manipulation Language (DML)
- ▶ Data Query Language (DQL)
- ▶ Data Control Language (DCL)
- ▶ Transaction Control Language (TCL)

Data Definition Language (DDL)

- ▶ Used to define and modify database structures.
- ▶ Commands:
 - ▶ CREATE
 - ▶ ALTER
 - ▶ DROP
 - ▶ TRUNCATE
 - ▶ RENAME

Example:

- ▶ `CREATE TABLE Student ( ID INT, Name VARCHAR(50) );`
- ▶ `ALTER TABLE Student ADD (SALARY INT );`
- ▶ `ALTER TABLE Student MODIFY (SALARY NUMBER(10) );`
- ▶ `DROP TABLE Student;`
- ▶ `TRUNCATE TABLE Student;`
- ▶ `RENAME TABLE Student TO Students;`

Data Manipulation Language (DML)

- ▶ Used to insert, update, and delete data.
- ▶ Commands:
 - ▶ INSERT
 - ▶ UPDATE
 - ▶ DELETE
- ▶ **Example:**
- ▶ INSERT INTO Student VALUES (1, 'John');
- ▶ UPDATE Student SET Age = 22 WHERE StudentID = 101;
- ▶ DELETE FROM Student WHERE StudentID = 103;

Data Query Language (DQL)

- ▶ Used to retrieve data from databases.
- ▶ Main Command:
 - ▶ SELECT
- ▶ **Example:**
- ▶ `SELECT * FROM Student;`

Data Control Language (DCL)

- ▶ Controls access and permissions.
- ▶ Commands:
 - ▶ GRANT- Gives privileges to users.
 - ▶ REVOKE - Removes privileges from users.

Example:

- ▶ GRANT SELECT ON Student TO User1;
- ▶ REVOKE SELECT ON Student FROM User1;

Transaction Control Language (TCL)

- ▶ Manages database transactions.
- ▶ Commands:
 - ▶ COMMIT
 - ▶ ROLLBACK
 - ▶ SAVEPOINT
- ▶ **Example:**
- ▶ COMMIT;

Different Types of Database Users

- ▶ A Database User is defined as a person who interacts with data daily for updating, reading, and modifying the given data. Database users can access and retrieve data from the database through the Database Management System (DBMS) applications and interfaces.
- ▶ **Types of Database Users**
- ▶ Database users are categorized based on their interaction with the database. There are seven types of database users in DBMS. Below mentioned are the types of database users:
- ▶ **1. Database Administrator (DBA)**
- ▶ A Database Administrator (DBA) defines the schema and manages all three levels of the database architecture. They create user accounts, control access, and ensure data security by authorizing only trusted users. DBAs are also responsible for handling security breaches and optimizing system performance.
- ▶ DBA also monitors the recovery and backup and provides technical support.
- ▶ The DBA has a DBA account in the DBMS which is called a system or superuser account.
- ▶ DBA repairs damage caused due to hardware and/or software failures.
- ▶ DBA is the one having privileges to perform DCL (Data Control Language) operations such as GRANT and REVOKE, to allow/restrict a particular user from accessing the database.

► **2. Naive / Parametric End Users**

- Parametric End Users are the unsophisticated who lacks DBMS knowledge but they frequently use the database applications in their daily life to get the desired results. For example, Railway's ticket booking users, Clerks in any bank are naive user because they lacks DBMS knowledge but they still use the database.

► **3. A System Analyst**

- A system Analyst is a user who analyzes the requirements of parametric end users. They check whether all the requirements of end users are satisfied.

► **4. Sophisticated Users**

- Sophisticated users like engineers or analysts interact directly with the database using SQL queries. They don't write full application code but use the query processor to access and manipulate data. These users develop custom database applications based on their specific needs.

► **5. Database Designers**

- Database Designers create the structure of a database, including tables, views, indexes, triggers, and constraints. They decide what data to store and how it should be related based on user requirements. Their design is finalized before the database is built or populated with data.

► **6. Application Programmers**

- Application Programmers, also known as System Analysts or Software Engineers, write backend code for application programs. They use languages like C, Visual Basic, COBOL, etc., to design, test, and maintain programs. These programs, called "canned transactions," help naive users interact with the database efficiently.

► **7. Casual Users / Temporary Users**

- Casual Users are the users who occasionally use/access the database but each time when they access the database they require the new information, for example, Middle or higher level manager.

► **8. Specialized users**

- Specialized users are sophisticated users who write specialized database application that does not fit into the traditional data-processing framework. Among these applications are computer aided-design systems, knowledge-base and expert systems etc.

Phases of Database Design in DBMS

1. Requirements Analysis (Defining Business Needs)

- ▶ In this first phase of the database design process in DBMS, the goal is to gather requirements from stakeholders, users, and business managers. This includes understanding the scope of the system and identifying what data needs to be captured, stored, and processed. For example, if the database is for a hospital management system, this step would identify the need to capture data on patients, doctors, treatments, and appointments.
- ▶ This is a critical step because it sets the foundation for the entire design. If the requirements are misunderstood or poorly documented, the entire system can fail. In the context of a hospital system, not capturing critical patient information would be disastrous.

Conceptual Design (ER Diagram and Entity Modeling)

- ▶ Once the requirements are defined, the conceptual design phase begins. This involves creating a high-level database design and ER diagram. An ER (Entity-Relationship) diagram visually represents the entities (like Customers, Orders, Products) and their relationships. The ER model is used to understand the data and how different entities are related.
- ▶ For example, in an e-commerce database, entities like "Customer," "Product," and "Order" will be represented as boxes, while their relationships (e.g. Customers place Orders) will be represented with lines connecting them.

Logical Database Design (Normalization and Schema Development)

- ▶ Once the ER diagram is complete, we move on to logical database design. This phase is concerned with converting the conceptual design into a logical structure, focusing on how the data will be organized in tables and defining relationships between them.
- ▶ In relational database design, this means creating tables and relationships using primary and foreign keys. The goal is to ensure that the data is stored efficiently, without unnecessary duplication. For instance, instead of having a customer's details (name, email) stored in multiple orders, we store the customer data in one table and reference it in the orders table.

Example: Relational Model for E-commerce

- ▶ Customer Table: CustomerID, Name, Email, Address
- ▶ Product Table: Product ID, Product Name, Category

Schema Refinement in Database Design (Normalization)

- ▶ Schema refinement in database design refers to improving the database structure to eliminate redundancy and ensure that data is logically stored. The most widely used technique is normalization, which involves organizing the data into tables and removing anomalies (insertion, update, and deletion anomalies).
- ▶ For instance, if a hospital's database stores both patient and doctor information in the same table, updating a doctor's name would require changes in multiple rows, leading to errors. To avoid this, we split the data into separate tables for doctors and patients, and link them through foreign keys. Normalization typically involves the first three normal forms (1NF, 2NF, 3NF), where each successive form removes specific kinds of redundancy and dependency.

Physical Design

- ▶ After the logical structure is established, the next phase is physical database design. Here, decisions are made about how the data will be stored on physical storage media, like servers or cloud databases. Indexes, file organization, and access methods are defined during this phase. The goal is to optimize data retrieval, ensuring that the database is both efficient and scalable.
- ▶ For example, for a large e-commerce platform, creating indexes on frequently searched fields like "ProductName" or "Category" can speed up search queries. This ensures that users get results faster when searching for products.
- ▶

Testing and Implementation

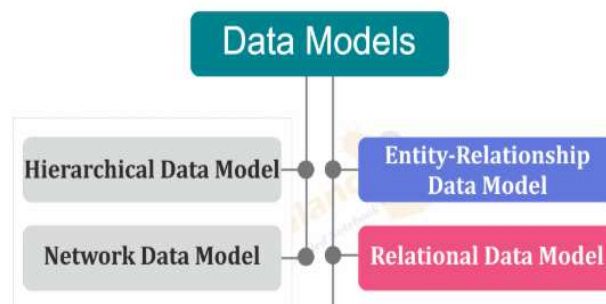
- After the database design is finalized, it is implemented and tested. During testing, the design is validated by running queries and performing CRUD (Create, Read, Update, Delete) operations to ensure that everything works as expected.

Data Models

- ▶ A data model is a collection of high-level data description constructs that hide many low-level storage details. A DBMS allows a user to define the data to be stored in terms of a data model.

The data models can be classified into different categories:

- ▶ **Hierarchical Model**
- ▶ **Network Model**
- ▶ **Entity-Relationship Model**
- ▶ **Relational Model**



1. Hierarchical Model

- ▶ Hierarchical Model was the first DBMS model. This model organises the data in the hierarchical tree structure.
- ▶ The hierarchy starts from the root which has root data and then it expands in the form of a tree adding child node to the parent node. This model easily represents some of the real-world relationships like food recipes, sitemap of a website etc.

Disadvantages of the hierarchical model:

- ▶ easy to design at the same time it is quite complex to implement.
- ▶ This model also lacks flexibility as the changes in the new tables or segments often yield very complex system management tasks. Here, a deletion of one segment can lead to the involuntary deletion of all segments under it.

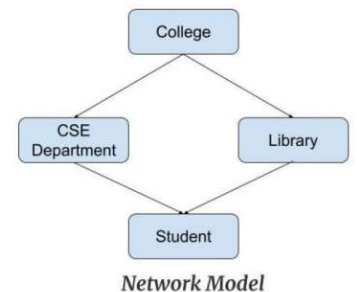


Network Model

- ▶ This model is an extension of the hierarchical model, the only difference is that a record can have more than one parent. It replaces the hierarchical tree with a graph.
- ▶ The network model was created to represent complex data relationships more effectively when compared to hierarchical models, to improve database performance and standards.
- ▶ Depicts both one-to-many (1:M) and many-to-many (M:N) relationships.

Disadvantages of the Network Model:

- ▶ Database contains a complex array of pointers.
- ▶ System complexity limits efficiency.
- ▶ Structural changes require changes in all application programs.
- ▶ Navigation systems yield complex implementation and management.
- ▶ Keep heavy pressure on programmers due to the complex structure.
- ▶ Any change like updating, deletion, insertion is very complex.

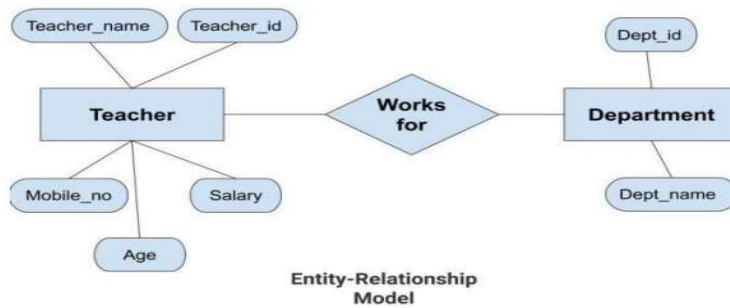


Entity-Relationship Model

- An ER model is the logical representation of data as objects and relationships among them. These objects are known as entities, and relationship is an association among these entities.

Entity-Relationship Model Components

- ER diagram basically having three components:
- **Entities** – It is a real-world thing which can be a person, place, or even a concept. For Example: Department, Admin, Courses, Teachers, Students, Building, etc are some of the entities of a School Management System.
- **Attributes** – An entity which contains a real-world property called an attribute. For Example: The entity employee has the property like employee id, salary, age, etc.
- **Relationship** – Relationship tells how two attributes are related. For Example: Employee works for a department.



► **Advantages of Entity-Relationship Model**

- The ER model is easy to build.
- This model is widely used by database designers for communicating their ideas.
- This model can easily convert to any other model like network model, hierarchical model etc.
- It is integrated with the dominant relational model.

► **Disadvantages of Entity-Relationship Model**

- There is no industry standard for developing an ER model.
- Information might be lost or hidden in the ER model.
- There is no Data Manipulation
- There is limited relationship representation.

Relational Model

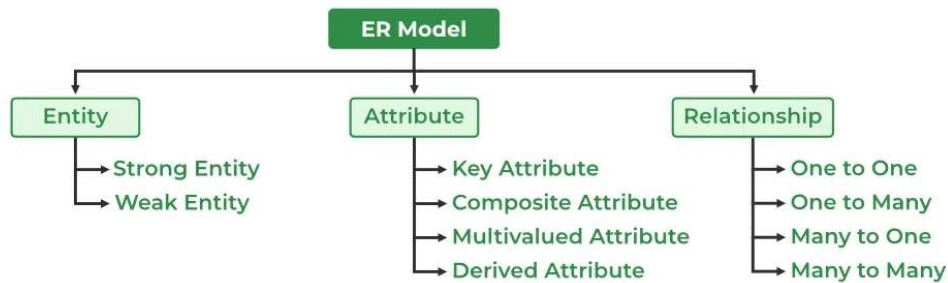
- The relational model uses a collection of tables to represent both data and the relationships. Tables are also known as relations. Each table has multiple columns represent as attributes, Attributes are the properties which define a relation. Each row of the table represents as Tuple, Tuple is one piece of information.

Emp_id	Emp_name	Job_name	Salary	Mobile_no	Dep_id	Project_id
AfterA001	John	Engineer	100000	9111037890	2	99
AfterA002	Adam	Analyst	50000	9587569214	3	100
AfterA003	Kande	Manager	890000	7895212355	2	65

EMPLOYEE TABLE

Introduction of ER Model

- The Entity-Relationship Model (ER Model) is a conceptual model for designing a database. This model represents the logical structure of a database, including entities, their attributes, and relationships between them.
- **Entity:** An object that is stored as data. E.g: *Student*, *Course*, or *Company*.
- **Attribute:** Properties that describe an entity. E.g: *StudentID*, *CourseName*, or *EmployeeEmail*.
- **Relationship:** A connection between entities. E.g: *Student* enrolls in a *Course*.

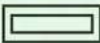


ER Diagrams in DBMS

- ▶ ER diagrams represent the E-R model in a database, making them easy to convert into relations (tables).
- ▶ These diagrams serve the purpose of real-world modeling of objects which makes them intently useful.
- ▶ Unlike technical schemas, ER diagrams require no technical knowledge of the underlying DBMS used.
- ▶ They visually model data and its relationships, making complex systems easier to understand.

Symbols Used in ER Model

- ER Model is used to model the logical view of the system from a data perspective which consists of these symbols:

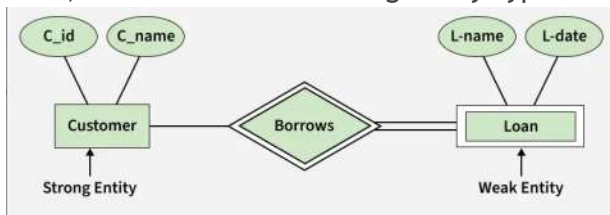
Figures	Symbols	Represents
Rectangle		Entities in ER Model
Ellipse		Attributes in ER Model
Diamond		Relationships among Entities
Line		Attributes to Entities and Entity Sets with Other Relationship Types
Double Ellipse		Multi-Valued Attributes
Double Rectangle		Weak Entity

Entity

- ▶ It represents a real-world object, concept or thing about which data is stored in a database. It act as a building block of a database. Tables in relational database represent these entities.
- ▶ Example of entities:
- ▶ **Real-World Objects:** Person, Car, Employee etc.
- ▶ **Concepts:** Course, Event, Reservation etc.
- ▶ **Things:** Product, Document, Device etc.
- ▶ The entity type defines the structure of an entity, while individual instances of that type represent specific entities.

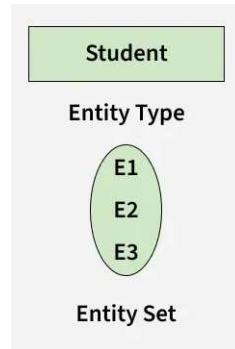
Types of Entity

- ▶ There are two main types of entities:
- ▶ **1. Strong Entity**
- ▶ A type of entity that has a key Attribute that can uniquely identify each instance of the entity. A **Strong Entity** does not depend on any other Entity in the Schema for its identification. It has a primary key that ensures its uniqueness and is represented by a rectangle in an ER diagram.
- ▶ **2. Weak Entity**
- ▶ It cannot be uniquely identified by its own attributes alone. It depends on a strong entity to be identified. A weak entity is associated with an identifying entity (strong entity), which helps in its identification. A weak entity are represented by a double rectangle. The participation of weak entity types is always total. The relationship between the weak entity type and its identifying strong entity type is called identifying relationship and it is represented by a double diamond.
- ▶ **Example:**
- ▶ A company may store the information of dependents (Parents, Children, Spouse) of an Employee. But the dependents can't exist without the employee. So dependent will be a Weak Entity Type and Employee will be identifying entity type for dependent, which means it is Strong Entity Type.



Entity Set

- Refers to an individual object of an entity type, and the collection of all entities of a particular type is called an entity set. For example, E1 is an entity that belongs to the entity type "Student," and the group of all students forms the entity set.



- We can represent the entity sets in an ER Diagram but we can't represent individual entities because an entity is like a row in a table, and an ER diagram shows the structure and relationships of data, not specific data entries (like rows and columns). An ER diagram is a visual representation of the data model, not the actual data itself.

Attributes in ER Model

- Attributes are the properties that define the entity type. For example, for a Student entity Roll_No, Name, DOB, Age, Address, and Mobile_No are the attributes that define entity type Student. In ER diagram, the attribute is represented by an oval.



- **Types of Attributes**

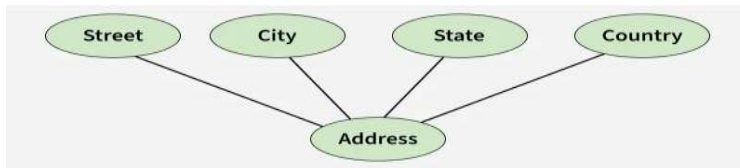
- **1. Key Attribute**

- The attribute which uniquely identifies each entity in the entity set is called the key attribute. For example, Roll_No will be unique for each student. In ER diagram, the key attribute is represented by an oval with an underline.



Composite Attribute

- ▶ An attribute composed of many other attributes is called a composite attribute. For example, the Address attribute of the student Entity type consists of Street, City, State, and Country. In ER diagram, the composite attribute is represented by an oval comprising of ovals.

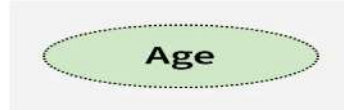


- ▶ **Multivalued Attribute**
- ▶ An attribute consisting of more than one value for a given entity. For example, Phone_No (can be more than one for a given student). In ER diagram, a multivalued attribute is represented by a double oval.

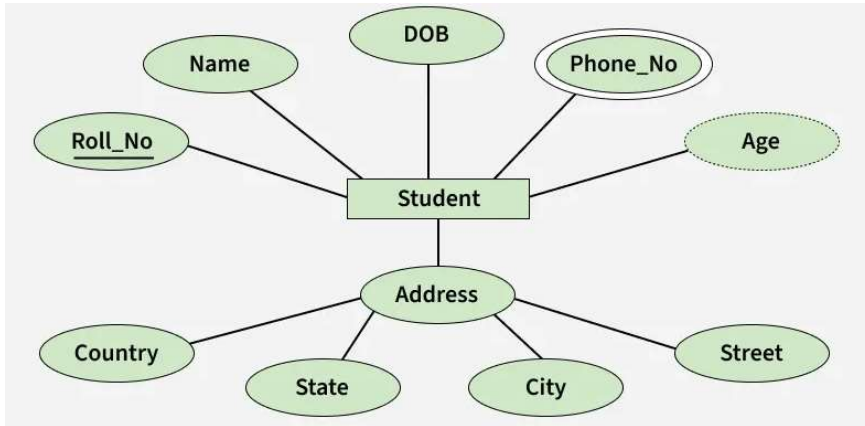


Derived Attribute

- An attribute that can be derived from other attributes of the entity type is known as a derived attribute. e.g.; Age (can be derived from DOB). In ER diagram, the derived attribute is represented by a dashed oval.



- The Complete Entity Type Student with its Attributes can be represented as:



Degree of a Relationship Set

- The number of entity sets that participate in a relationship set is termed as the degree of that relationship set.

Types of Relationship Sets

- On the basis of degree of a relationship set, a relationship set can be classified into the following types-

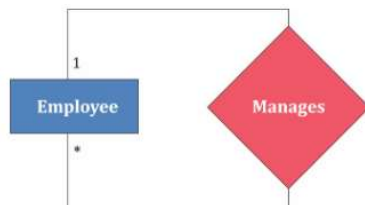
1. Unary relationship set

2. Binary relationship set

3. Ternary relationship set

Unary Relationship Set

- Unary relationship set is a relationship set where only one entity set participates in a relationship set.



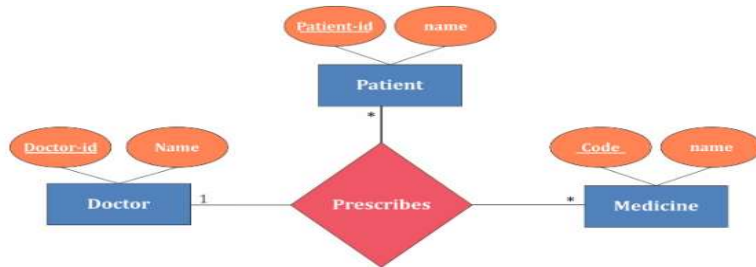
Binary Relationship Set

Binary relationship set is a relationship set where two entity sets participate in a relationship set.



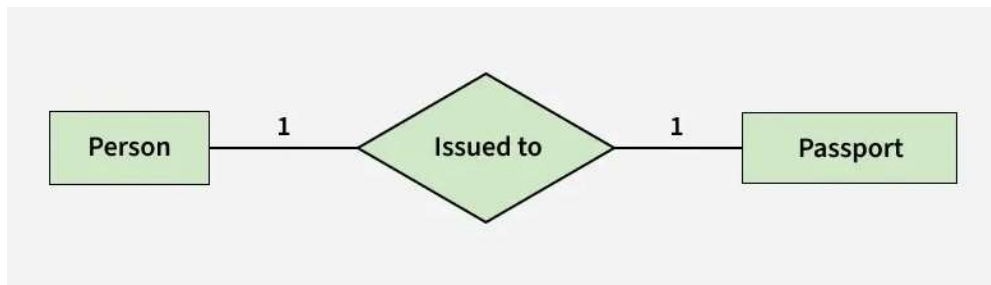
► Ternary Relationship Set

- Ternary relationship set is a relationship set where three entity sets participate in a relationship set.



Cardinality in ER Model

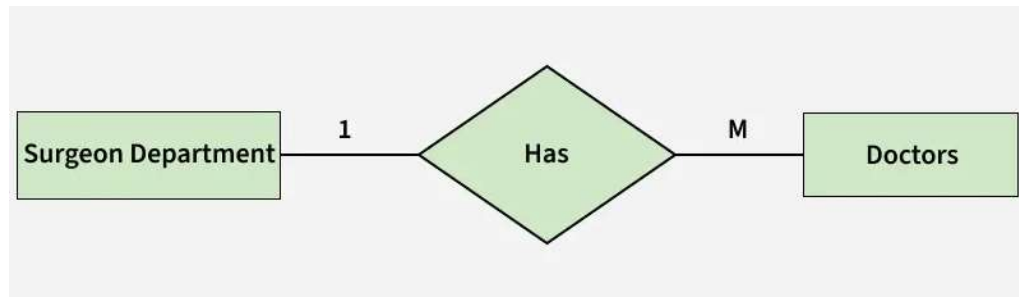
- ▶ The maximum number of times an entity of an entity set participates in a relationship set is known as cardinality.
- ▶ Cardinality can be of different types:
- ▶ **1. One-to-One**
- ▶ When each entity in each entity set can take part only once in the relationship, the cardinality is one-to-one. Let us assume that one person can be issued only one passport, and one passport is issued to only one person. So, the relationship will be One-to-One (1 : 1), meaning that each person has a single passport, and each passport belongs to a single person.



One-to-Many

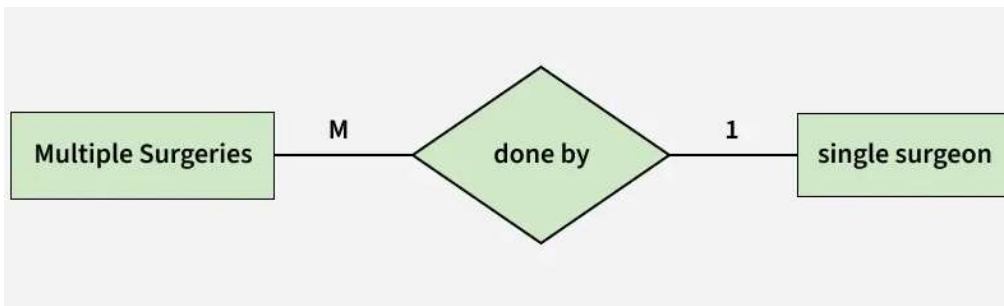
- In a one-to-many relationship, one entity can be associated with multiple entities. For example, a single Surgeon Department can have many Doctors. Therefore, the cardinality of this relationship is 1 to M, meaning one department can have many doctors.

►



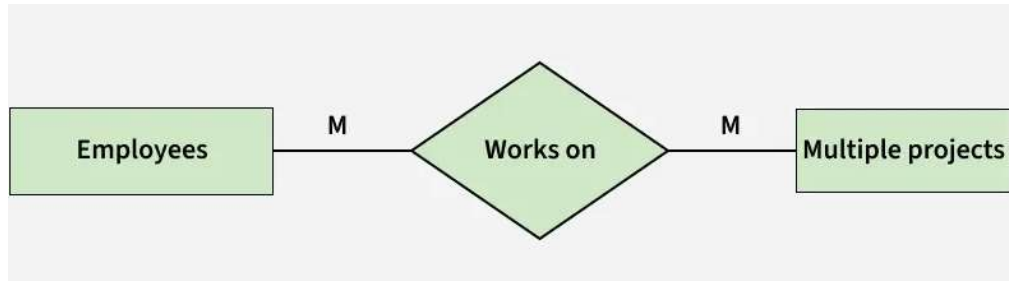
Many-to-One

- ▶ When entities in one entity set can take part only once in the relationship set and entities in other entity sets can take part more than once in the relationship set, cardinality is many to one.
- ▶ Let us assume that multiple surgeries can be performed by one surgeon, but one surgery is performed by only one surgeon. So, the cardinality will be M to 1, meaning that many surgeries can be done by a single surgeon, but each surgery is done by only one surgeon.



Many-to-Many

- When entities in all entity sets can take part more than once in the relationship cardinality is many to many. Let us assume that an employee can work on multiple projects and each project can have multiple employees working on it. So, the relationship will be many-to-many (M:N), meaning that one employee may be associated with several projects, and one project may involve several employees.



Relational Constraints

- Restrictions or sets of rules imposed on the database contents. It validates the quality of the database. It validates the various operations like data insertion, updation, and other processes that have to be performed without affecting the integrity of the data. Mainly Constraints on the relational database are of 4 types

- 1.Domain constraints
- 2.Key constraints or Uniqueness Constraints
- 3.Entity Integrity constraints
- 4.Referential integrity constraints

1. Domain Constraints

- ▶ Every domain must contain atomic values(smallest indivisible units) which means composite and multi-valued attributes are not allowed.
- ▶ We perform a datatype check here, which means when we assign a data type to a column we limit the values that it can contain. Eg. If we assign the datatype of attribute age as int, we can't give it values other than int datatype.
- ▶ **Example:**

EID	Name	Phone
01	Bikash Dutta	123456789
		234456678

2. Key Constraints or Uniqueness Constraints

- ▶ These are called uniqueness constraints since it ensures that every tuple in the relation should be unique.
- ▶ A relation can have multiple keys or candidate keys(minimal superkey), out of which we choose one of the keys as the primary key, we don't have any restriction on choosing the primary key out of candidate keys, but it is suggested to go with the candidate key with less number of attributes.
- ▶ Null values are not allowed in the primary key, hence Not Null constraint is also part of the key constraint.

Example:

EID	Name	Phone
01	Bikash	6000000009
02	Paul	9000090009
01	Tuhin	9234567892

3. Entity Integrity Constraints

- ▶ Entity Integrity constraints say that no primary key can take a NULL value, since using the primary key we identify each tuple uniquely in a relation.
- ▶ Example:

EID	Name	Phone
01	Bikash	9000900099
02	Paul	600000009
NULL	Sony	9234567892

4. Referential Integrity Constraints

- ▶ The Referential integrity constraint is specified between two relations or tables and used to maintain the consistency among the tuples in two relations.
- ▶ This constraint is enforced through a foreign key, when an attribute in the foreign key of relation R1 has the same domain(s) as the primary key of relation R2, then the foreign key of R1 is said to reference or refer to the primary key of relation R2.
- ▶ The values of the foreign key in a tuple of relation R1 can either take the values of the primary key for some tuple in relation R2, or can take NULL values, but can't be empty.
- ▶ **Example:**

EID	Name	DNO
01	Divine	12
02	Dino	22
04	Vivian	14

DNO	Place
12	Jaipur
13	Mumbai
14	Delhi

Constraints

- ▶ In a **DBMS (Database Management System)**, **constraints** are rules applied to table columns to ensure the accuracy, consistency, and integrity of the data stored in the database.
- ▶ **Types of Constraints in DBMS**
- ▶ **NOT NULL**
 - ▶ Ensures that a column cannot have a NULL value.
 - ▶ Example: `CREATE TABLE Student ( StudentID INT, Name VARCHAR(50) NOT NULL );`
 - ▶ Here, every student must have a name.
- ▶ **UNIQUE**
 - ▶ Ensures that all values in a column are different.
 - ▶ Example: `CREATE TABLE Student ( Email VARCHAR(100) UNIQUE );`
 - ▶ No two students can have the same email address.

► **PRIMARY KEY**

- Uniquely identifies each record in a table.
- A primary key cannot contain NULL values and must be unique.
- Example:CREATE TABLE Student (StudentID INT PRIMARY KEY, Name VARCHAR(50));
- StudentID is unique for every student.

► **FOREIGN KEY**

- Creates a relationship between two tables.
- Ensures that values in one table exist in another table.
- Example:CREATE TABLE Department (DeptID INT PRIMARY KEY); CREATE TABLE Student (StudentID INT PRIMARY KEY, DeptID INT, FOREIGN KEY (DeptID) REFERENCES Department(DeptID));
- A student can only belong to an existing department.

► **CHECK**

- Ensures that values satisfy a specified condition.
- Example:CREATE TABLE Student (Age INT CHECK (Age >= 18));
- Only students aged 18 or above can be inserted.

► **DEFAULT**

- Assigns a default value if no value is provided.
- Example:CREATE TABLE Student (Country VARCHAR(30) DEFAULT 'India');
- If no country is specified, "India" is stored.

Keys in Relational Model

- ▶ Keys are fundamental elements of the relational database model that ensure uniqueness, data integrity, and efficient data access.
- ▶ They uniquely identify each row in a table.
- ▶ They prevent data duplication and maintain consistency.
- ▶ They create relationships between different tables.

Types of Database Keys

► 1. Primary Key

- Uniquely identifies each record in a table.
- Cannot contain NULL values.
- Must contain unique values.

► Example:

- Student(StudentID, Name, Age) StudentID → Primary Key

► 2. Candidate Key

- A column (or combination of columns) that can uniquely identify a record.
- A table can have multiple candidate keys.
- One candidate key is chosen as the primary key.

► Example:

- StudentID, Email Both can uniquely identify a student, so both are candidate keys.

► **3. Super Key**

- Any set of attributes that uniquely identifies a record.
- It may contain extra attributes.

► **Example:**

- {StudentID} {StudentID, Name} Both are super keys because they uniquely identify a student.

► **4. Alternate Key**

- Candidate keys that are **not** selected as the primary key.

► **Example:**

- Candidate Keys: StudentID, Email Primary Key: StudentID Alternate Key: Email

► **5. Foreign Key**

- A key in one table that refers to the primary key of another table.

- Used to maintain relationships between tables.

► **Example:**

- Department ----- DeptID (Primary Key) Student ----- StudentID DeptID (Foreign Key)

► **6. Composite Key (Compound Key)**

- A key formed by combining two or more columns.

- Used when a single column cannot uniquely identify a record.

► **Example:**

- Enrollment(StudentID, CourseID) The combination of StudentID and CourseID uniquely identifies each enrollment.

► 5. Foreign Key

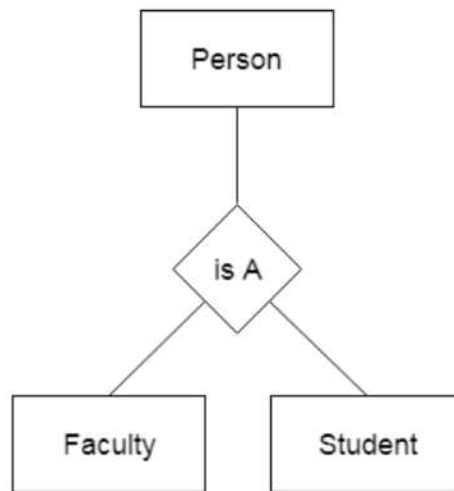
- A key in one table that refers to the primary key of another table.
- Used to maintain relationships between tables.
- **Example:**
- Department ----- DeptID (Primary Key) Student ----- StudentID DeptID (Foreign Key)

► 6. Composite Key (Compound Key)

- A key formed by combining two or more columns.
- Used when a single column cannot uniquely identify a record.
- **Example:**
- Enrollment(StudentID, CourseID) The combination of StudentID and CourseID uniquely identifies each enrollment.

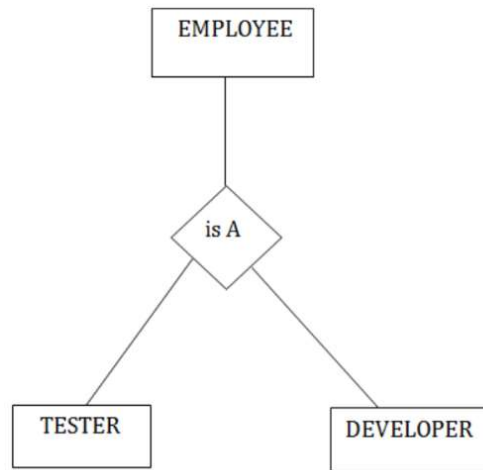
Generalization

- **Generalization** is a bottom-up approach in data modeling where multiple lower-level entities (subclasses) are combined into a higher-level, more generalized entity (superclass). Generalization is used when you have entities that share common attributes and relationships, and you want to group them into a generalized, abstract entity.



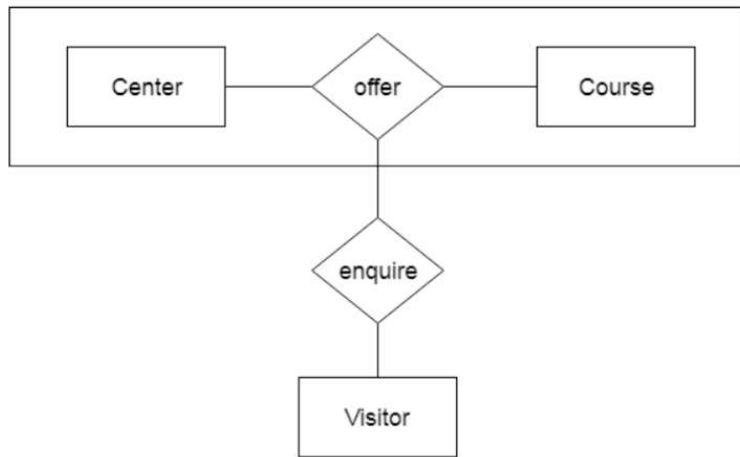
Specialization

- **Specialization** is the opposite of **generalization**. It is a top-down approach in data modeling where a higher-level, more generalized entity (superclass) is divided into multiple lower-level, more specific entities (subclasses). Specialization allows us to capture differences between entities that share some common characteristics but have unique attributes or behaviors.

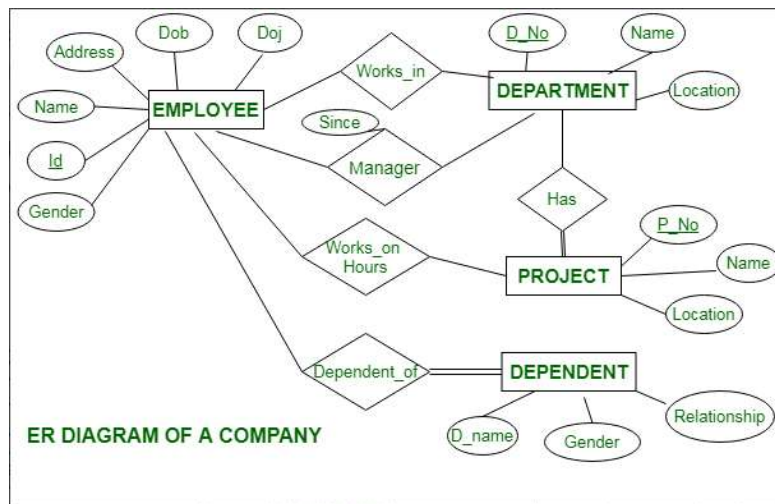


Aggregation

- **Aggregation** is a concept used when a relationship involves an entity set that itself is a complex entity or a collection of other entities. Aggregation is used to simplify the model when relationships between entities are complex, and we need to treat a relationship set as a higher-level entity. It allows us to treat a relationship as an entity and perform higher-level operations on it.



conceptual design with the E-R model for large enterprise. in dbms



Thank you